

```
1 from solution import *
2 from helper import *
3
4
5 if __name__ == '__main__':
6
7     data_dir = '../cifar-10-batches-py/'
8
9     print('Loading and preprocessing...')
10    x_train, y_train, x_test, y_test = load_data(data_dir)
11    #set 'normalize' to false for test 2
12    x_train, x_test = preprocess(x_train, x_test, normalize=True)
13    x_train, y_train, x_valid, y_valid = train_valid_split(x_train, y_train)
14
15    model = LeNet_Cifar10(n_classes=10)
16    model.train(x_train, y_train, x_valid, y_valid, 128, 20)
17
18    accuracy = model.test(x_test, y_test)
19    with open('test_result.txt', 'w') as f:
20        f.write(str(accuracy))
21
22    print('Test accuracy: %.4f' %accuracy)
```

```
1 import os
2 import pickle
3 import numpy as np
4
5
6 def train_valid_split(x_train, y_train, split_index=45000):
7     """Split the original training data into a new training dataset
8     and a validation dataset.
9
10    Args:
11        x_train: An array of shape [50000, 32, 32, 3].
12        y_train: An array of shape [50000,].
13        split_index: An integer.
14
15    Returns:
16        x_train_new: An array of shape [split_index, 32, 32, 3].
17        y_train_new: An array of shape [split_index,].
18        x_valid: An array of shape [50000-split_index, 32, 32, 3].
19        y_valid: An array of shape [50000-split_index,].
20    """
21    x_train_new = x_train[:split_index]
22    y_train_new = y_train[:split_index]
23    x_valid = x_train[split_index:]
24    y_valid = y_train[split_index:]
25
26    return x_train_new, y_train_new, x_valid, y_valid
27
28
```

```
1  Instruction of programming assignments for CSCE421: Machine Learning
2
3
4  Environment Building
5  -----
6  Please install python packages tqdm using:
7  "pip install tqdm" or
8  "conda install tqdm" (for Anaconda environment)
9
10
11 Installation of PyTorch
12 -----
13 If you are using Anaconda environment, simply using:
14 "conda install -c pytorch pytorch"
15
16 For other environments, using:
17 "pip install torch"
18
19 You can refer to https://pytorch.org/get-started/locally/ for more details.
20
21 Note: by default, you are installing the CPU version of PyTorch. You are not required
22 to install the GPU version.
23
24
25 Dataset Descriptions
26 -----
27 We will use the Cifar-10 Dataset to do the image classification. The
28 CIFAR-10 dataset consists of 60000 32x32 colour images in 10 classes,
29 with 6000 images per class. There are 50000 training images and 10000
30 test images.
31
32 Download the CIFAR-10 python version from the website:
33 https://www.cs.toronto.edu/~kriz/cifar.html
34 then extract the files.
35
36 Follow the instructions in "Dataset layout" to load the data and labels.
37 The training data and labels are save(d) in 5 files. You will need to integrate
38 the 5 parts. The image data in each file is a 10000x3072 numpy array. Each
39 row of the array stores a 32x32 colour image. The first 1024 entries contain
40 the red channel values, the next 1024 the green, and the final 1024 the blue.
41
42
43 Assignment Descriptions
44 -----
45 There are total three Python files including 'main.py', 'solution.py' and
46 'helper.py'. In this assignment, you only need to add your solution in
47 'solution.py' file following the given instruction. However, you might need
48 to read all the files to fully understand the requirement.
49
50 The 'helper.py' includes all the helper functions for the assignments.
51 The 'main.py' is used to test your solution.
52
53 Notes: Do not change anything in 'main.py' and 'helper.py' files. Only try
```

```
54 to add your code to 'solution.py' file and keep function names and parameters
55 unchanged.
56
57
58 APIs you will need
59 -----
60 torch.nn.Conv2d
61 torch.nn.MaxPool2d
62 torch.nn.Linear
63 torch.nn.ReLU
64 torch.nn.BatchNorm2d
65 torch.nn.BatchNorm1d
66 torch.nn.Dropout
67
68 Refer to https://pytorch.org/docs/stable/nn.html for more details.
```

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  import numpy as np
5  import pickle, tqdm, os
6  import time
7
8
9  def load_data(data_dir):
10     """
11     To load the Cifar-10 Dataset from files and reshape the
12     images arrays from shape [3072,] to shape [3, 32, 32].
13
14     Please follow the instruction on how to load the data and
15     labels at https://www.cs.toronto.edu/~kriz/cifar.html
16
17     Args:
18         data_dir: String. The directory where data batches are
19                 stored.
20
21     Returns:
22         x_train: An numpy array of shape [50000, 3, 32, 32].
23                 (dtype=np.uint8)
24         y_train: An numpy array of shape [50000,].
25                 (dtype=np.int64)
26         x_test: An numpy array of shape [10000, 3, 32, 32].
27                (dtype=np.uint8)
28         y_test: An numpy array of shape [10000,].
29                (dtype=np.int64)
30     """
31
32     ### YOUR CODE HERE
33     # Initialize arrays for training data
34     x_train = []
35     y_train = []
36
37     # Load the 5 training batches
38     for i in range(1, 6):
39         file_path = os.path.join(data_dir, f'data_batch_{i}')
40         with open(file_path, 'rb') as f:
41             batch = pickle.load(f, encoding='bytes')
42
43         if i == 1:
44             x_train = batch[b'data']
45             y_train = batch[b'labels']
46         else:
47             x_train = np.vstack((x_train, batch[b'data']))
48             y_train = np.concatenate((y_train, batch[b'labels']))
49
50     # Load test data
51     test_file_path = os.path.join(data_dir, 'test_batch')
52     with open(test_file_path, 'rb') as f:
53         test_batch = pickle.load(f, encoding='bytes')

```

```

54
55     x_test = test_batch['data']
56     y_test = test_batch['labels']
57
58     # Reshape from [N, 3072] to [N, 3, 32, 32]
59     x_train = x_train.reshape(-1, 3, 32, 32)
60     x_test = x_test.reshape(-1, 3, 32, 32)
61
62     # Convert labels to int64
63     y_train = np.array(y_train, dtype=np.int64)
64     y_test = np.array(y_test, dtype=np.int64)
65
66     ### END YOUR CODE
67
68     return x_train, y_train, x_test, y_test
69
70
71 def preprocess(train_images, test_images, normalize=False):
72     """
73     To preprocess the data by
74     (1).Rescaling the pixels from integers in [0,255) to
75     floats in [0,1), or
76     (2).Normalizing each image using its mean and variance.
77
78     Args:
79     train_images: An numpy array of shape [50000, 3, 32, 32].
80     (dtype=np.uint8)
81     test_images: An numpy array of shape [10000, 3, 32, 32].
82     (dtype=np.uint8)
83     normalize: Boolean. To control to rescale or normalize
84     the images.
85
86     Returns:
87     train_images: An numpy array of shape [50000, 3, 32, 32].
88     (dtype=np.float64)
89     test_images: An numpy array of shape [10000, 3, 32, 32].
90     (dtype=np.float64)
91     """
92     ### YOUR CODE HERE
93     # Convert to float64
94     train_images = train_images.astype(np.float64)
95     test_images = test_images.astype(np.float64)
96
97     if normalize:
98         # Normalize each image using its mean and variance
99         for i in range(train_images.shape[0]):
100             for c in range(3): # For each color channel
101                 mean = np.mean(train_images[i, c, :, :])
102                 std = np.std(train_images[i, c, :, :])
103                 if std > 0: # Avoid division by zero
104                     train_images[i, c, :, :] = (train_images[i, c, :, :] - mean) / std
105
106         for i in range(test_images.shape[0]):

```

```

107         for c in range(3):
108             mean = np.mean(test_images[i, c, :, :])
109             std = np.std(test_images[i, c, :, :])
110             if std > 0:
111                 test_images[i, c, :, :] = (test_images[i, c, :, :] - mean) / std
112         else:
113             # Rescale pixel values from [0, 255] to [0, 1]
114             train_images /= 255.0
115             test_images /= 255.0
116
117         ### END CODE HERE
118         return train_images, test_images
119
120
121     class LeNet(nn.Module):
122         """
123         Build the LeCun network according to the architecture in the homework part 4(c)
124
125         You are free to use the listed APIs from torch.nn:
126         torch.nn.Conv2d
127         torch.nn.MaxPool2d
128         torch.nn.Linear
129         torch.nn.ReLU (or other activations)
130         torch.nn.BatchNorm2d
131         torch.nn.BatchNorm1d
132         torch.nn.Dropout
133
134         Refer to https://pytorch.org/docs/stable/nn.html
135         for the instructions for those APIs
136         """
137         def __init__(self, n_classes=None):
138             super(LeNet, self).__init__()
139             """
140             Define each layers of the model in __init__() function
141             """
142
143             ### YOUR CODE HERE
144             super(LeNet, self).__init__()
145
146             # First convolutional block
147             self.conv1 = nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5,
padding=2)
148             self.bn1 = nn.BatchNorm2d(6)
149             self.relu = nn.ReLU()
150             self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
151
152             # Second convolutional block
153             self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5)
154             self.bn2 = nn.BatchNorm2d(16)
155
156             # Feature map size calculation:
157             # Input: 32x32x3
158             # After conv1 (with padding=2): 32x32x6

```

```

159     # After pool1: 16x16x6
160     # After conv2 (no padding): 12x12x16
161     # After pool2: 6x6x16
162
163     # Fully connected layers
164     self.fc1 = nn.Linear(16 * 6 * 6, 120)
165     self.bn3 = nn.BatchNorm1d(120)
166
167     self.fc2 = nn.Linear(120, 84)
168     self.bn4 = nn.BatchNorm1d(84)
169
170     self.dropout = nn.Dropout(p=0.5)
171
172     self.fc3 = nn.Linear(84, n_classes)
173
174     ### END CODE HERE
175
176     def forward(self, x):
177         '''
178         Run forward pass of the model defined in the above __init__() function
179         Args:
180             x: Tensor of shape [None, 3, 32, 32]
181             for input images.
182
183         Returns:
184             logits: Tensor of shape [None, n_classes].
185         '''
186
187         ### YOUR CODE HERE
188         # First convolutional block
189         x = self.conv1(x)
190         #comment out these for experiment 2
191         #x = self.bn1(x)
192         x = self.relu(x)
193         x = self.pool(x)
194
195         # Second convolutional block
196         x = self.conv2(x)
197         #x = self.bn2(x)
198         x = self.relu(x)
199         x = self.pool(x)
200
201         # Flatten feature maps
202         x = x.reshape(-1, 16 * 6 * 6)
203
204         # First fully connected block
205         x = self.fc1(x)
206         #x = self.bn3(x)
207         x = self.relu(x)
208
209         # Second fully connected block
210         x = self.fc2(x)
211         #x = self.bn4(x)

```

```

212     x = self.relu(x)
213
214     # Dropout
215     #run1 include this, run 2 exclude this
216     #x = self.dropout(x)
217
218     #run2
219     #None
220
221
222     # Output layer
223     x = self.fc3(x)
224
225     ### END CODE HERE
226     return x
227
228
229 class LeNet_Cifar10(nn.Module):
230     def __init__(self, n_classes):
231
232         super(LeNet_Cifar10, self).__init__()
233
234         self.n_classes = n_classes
235         self.model = LeNet(n_classes=n_classes)
236         self.criterion = nn.CrossEntropyLoss()
237         self.optimizer = optim.Adam(self.model.parameters(), lr=0.001)
238
239     def train(self, x_train, y_train, x_valid, y_valid, batch_size, max_epoch):
240
241         num_samples = x_train.shape[0]
242         num_batches = int(num_samples / batch_size)
243
244         num_valid_samples = x_valid.shape[0]
245         num_valid_batches = (num_valid_samples - 1) // batch_size + 1
246
247         x_train = torch.from_numpy(x_train).float()
248         y_train = torch.from_numpy(y_train)
249         x_valid = torch.from_numpy(x_valid).float()
250         y_valid = torch.from_numpy(y_valid)
251
252         print('---Run...')
253         for epoch in range(1, max_epoch + 1):
254             self.model.train()
255             # To shuffle the data at the beginning of each epoch.
256             shuffle_index = np.random.permutation(num_samples)
257             curr_x_train = x_train[shuffle_index]
258             curr_y_train = y_train[shuffle_index]
259
260             # To start training at current epoch.
261             loss_value = []
262             qbar = tqdm.tqdm(range(num_batches))
263             for i in qbar:
264                 batch_start_time = time.time()

```

```

265
266         start = batch_size * i
267         end = batch_size * (i + 1)
268         x_batch = curr_x_train[start:end]
269         y_batch = curr_y_train[start:end]
270
271         self.optimizer.zero_grad()
272         outputs = self.model(x_batch)
273         loss = self.criterion(outputs, y_batch)
274         loss.backward()
275         self.optimizer.step()
276         if not i % 10:
277             qbar.set_description(
278                 'Epoch {:d} Loss {:.6f}'.format(
279                     epoch, loss))
280
281         # To start validation at the end of each epoch.
282         self.model.eval()
283         correct = 0
284         total = 0
285         print('Doing validation...', end=' ')
286         with torch.no_grad():
287             for i in range(num_valid_batches):
288
289                 start = batch_size * i
290                 end = min(batch_size * (i + 1), x_valid.shape[0])
291                 x_valid_batch = x_valid[start:end]
292                 y_valid_batch = y_valid[start:end]
293
294                 outputs = self.model(x_valid_batch)
295                 _, predicted = torch.max(outputs.data, 1)
296                 total += y_valid_batch.shape[0]
297                 correct += (predicted == y_valid_batch).sum().item()
298
299         acc = correct / total
300         print('Validation Acc {:.4f}'.format(acc))
301
302     def test(self, X_test, y_test):
303         self.model.eval()
304
305         X_test = torch.from_numpy(X_test).float()
306         y_test = torch.from_numpy(y_test)
307
308         accs = 0
309         for X, y in zip(X_test, y_test):
310
311             outputs = self.model(X.unsqueeze(0))
312             _, predicted = torch.max(outputs.data, 1)
313             accs += (predicted == y).sum().item()
314
315         accuracy = float(accs) / len(y_test)
316
317         return accuracy

```

1 0.6261